BÁO CÁO TIẾN ĐỘ THỰC HIỆN ĐỒ ÁN TỐT NGHIỆP

(Báo cáo tiến độ nghiên cứu tuần 7)

Sinh viên thực hiện: Cao Văn Mạnh

Lớp: K67M-AT

Tên đề tài: Nghiên cứu hệ thống thị giác máy tính thời gian thực cho robot trên nền tảng FPGA – Edge AI

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

Chương này trình bày cơ sở của phương pháp phát hiện hộp trên băng tải bằng đặc trưng HOG và bộ phân loại SVM tuyến tính, đồng thời giới thiệu nguyên lý thiết kế khối tiền xử lý ảnh trên FPGA. Nội dung tập trung vào ảnh số, lựa chọn vùng xử lý, học từ mẫu nền bị nhận nhầm, đánh giá bộ phát hiện và kiểm chứng khối chênh lệch pixel bằng Verilog. Đây là nền tảng cho việc xây dựng chương trình trên PC và mô phỏng phần cứng trong các chương tiếp theo.

2.1 Thị giác máy tính và bài toán phát hiện hộp

Thị giác máy tính nghiên cứu các phương pháp thu nhận và xử lý ảnh để trích xuất thông tin phục vụ một nhiệm vụ cụ thể. Trong bài toán băng tải, đầu vào là ảnh hoặc chuỗi khung hình, còn đầu ra của bộ phát hiện là vị trí các hộp dưới dạng khung bao hình chữ nhật và điểm đánh giá của từng dự đoán.

Cần phân biệt phân loại ảnh với phát hiện đối tượng. Phân loại xác định một vùng ảnh đã cho có chứa hộp hay không. Phát hiện đối tượng còn phải tìm vị trí và kích thước của hộp trong toàn cảnh. Vì vậy, một bộ phân loại tốt trên ảnh hộp đã cắt sẵn chưa bảo đảm bộ phát hiện sẽ định vị tốt trên ảnh băng tải.

Trong phạm vi đồ án, bài toán được xét với camera cố định và một vùng băng tải xác định. Chuỗi xử lý trên PC gồm tạo vùng ứng viên, chuẩn hóa vùng ảnh, trích xuất HOG, phân loại bằng SVM và loại các khung dự đoán trùng nhau. Khối phần cứng được nghiên cứu riêng để xác định mức thay đổi giữa hai pixel cùng vị trí của hai khung hình xám liên tiếp. Khối này cung cấp thông tin tiền xử lý; việc xác định đối tượng là hộp vẫn thuộc bộ phát hiện.

2.2 Ảnh số và vùng quan tâm

2.2.1 Biểu diễn ảnh xám 8 bit

Ảnh số được biểu diễn bằng một lưới pixel. Với ảnh xám 8 bit, mỗi pixel nhận giá trị nguyên từ 0 đến 255, trong đó 0 biểu diễn mức tối và 255 biểu diễn mức sáng. Ký hiệu $I_t(x,y)$ là giá trị mức xám tại tọa độ (x,y) của khung hình thứ t.

Một phép chuyển đổi thông dụng từ ảnh màu sang ảnh xám sử dụng tổng có trọng số của ba kênh màu:

$$I = 0.299R + 0.587G + 0.114B \quad (2.1)$$

Trong đó R, G và B lần lượt là giá trị kênh đỏ, lục và lam. Kết quả cần được làm tròn về kiểu dữ liệu của ảnh đầu ra. Khi sử dụng OpenCV, phải chú ý thứ tự kênh BGR của ảnh được đọc bằng cấu hình thông thường để chọn phép chuyển đổi phù hợp. [1]

2.2.2 Vùng quan tâm trên băng tải

Vùng quan tâm, gọi là ROI (Region of Interest), là phần ảnh được lựa chọn để xử lý. Trong cảnh camera cố định, ROI có thể giới hạn phạm vi tìm kiếm vào khu vực hộp di chuyển. Các vùng ngoài băng tải như tường, người hoặc kết cấu máy có thể được loại khỏi quá trình tạo ứng viên.

ROI giúp giảm số vùng cần đánh giá, nhưng cũng tạo ra giới hạn của bộ phát hiện: hộp nằm ngoài ROI có thể bị bỏ sót. Khi thay đổi vị trí camera, kích thước ảnh hoặc hướng băng tải, cần kiểm tra lại tọa độ ROI. Các kết quả so sánh quét toàn ảnh và quét ROI phải sử dụng cùng dữ liệu, cùng quy tắc đánh giá và cách xác định đối tượng thuộc phạm vi cần phát hiện.

2.3 Đặc trưng HOG

2.3.1 Nguyên lý

HOG (Histogram of Oriented Gradients) là đặc trưng mô tả phân bố hướng biến thiên cường độ ảnh trong các vùng nhỏ. Ý tưởng của HOG là biểu diễn cấu trúc hình dạng thông qua gradient, thay vì sử dụng trực tiếp toàn bộ giá trị pixel. Phương pháp được Dalal và Triggs trình bày trong nghiên cứu phát hiện người năm 2005. Khi áp dụng cho hộp, bộ phân loại cần được huấn luyện bằng dữ liệu hộp và nền của bài toán. [2]

Với ảnh xám I, gradient có thể được xấp xỉ bằng sai phân theo hai hướng:

$$\begin{aligned} G_x(x,y) &= I(x,+1y) - I(x,-1y) \\ G_y(x,y) &= I(x,y+1) - I(x,y-1) \end{aligned} \quad (2.2)$$

Độ lớn và hướng gradient được xác định bởi:

$$g(x,y) = \sqrt{G_x^2 + G_y^2},\ \theta(x,y) = \text{atan2}\big(G_y, G_x\big)\text{mod}\pi \ (2.3)$$

Hướng không dấu được biểu diễn trong khoảng từ 0 đến dưới 180°. Ảnh được chia thành các ô nhỏ gọi là cell. Trong mỗi cell, các pixel đóng góp vào histogram hướng với trọng số dựa trên độ lớn gradient. Các cell được nhóm thành block để chuẩn hóa cục bộ. Cuối cùng, đặc trưng của các block được ghép thành một vector cho vùng ảnh. [2]

2.3.2 Chuẩn hóa và kích thước đặc trưng

Một dạng chuẩn hóa block theo chuẩn L2 được viết như sau:

$$\mathbf{v} = \frac{\mathbf{h}}{\sqrt{\|\mathbf{h}\|_2^2 + \varepsilon^2}}$$

Trong đó h là vector histogram trước chuẩn hóa và ε là hằng số nhỏ để tránh chia cho 0. Chuẩn hóa L2-Hys thực hiện chuẩn hóa L2, giới hạn các thành phần lớn rồi chuẩn hóa lại. OpenCV cung cấp các tham số cửa sổ, cell, block, bước dịch block và số khoảng hướng để cấu hình HOG. [3]

Với cấu hình đang được sử dụng trong chương trình của đồ án, vùng ảnh đầu vào có kích thước 64 × 64 pixel, cell 8 × 8 pixel, block 16 × 16 pixel, bước dịch block 8 × 8 pixel và 9 khoảng hướng. Số block trên mỗi chiều là (64 − 16)/8 + 1 = 7. Mỗi block chứa 4 cell, do đó số thành phần của vector là:

$$D \ = \ 7 \ \times \ 7 \times 4 \times 9 \ = \ 1764 \ (2.5)$$

Đây là cấu hình của đồ án, cần được giữ thống nhất khi huấn luyện và suy luận. Việc đổi kích thước hoặc cách chuẩn hóa ảnh có thể làm thay đổi đặc trưng. HOG cũng không bảo đảm bất biến trước mọi góc nhìn, biến dạng hay che khuất; các trường hợp này cần được phản ánh trong dữ liệu huấn luyện và đánh giá.

2.4 Bộ phân loại SVM tuyến tính

2.4.1 Hàm quyết định và huấn luyện

SVM (Support Vector Machine) là phương pháp học có giám sát. Trong bài toán này, mỗi vùng ảnh được biểu diễn bằng vector HOG z và gán nhãn hộp hoặc nền. SVM tuyến tính xây dựng một siêu phẳng phân chia hai lớp, với hàm quyết định:

$$s(\mathbf{z}) = \mathbf{w}^\top \mathbf{z} + b \ (2.6)$$

Trong đó w là vector trọng số và b là hệ số tự do. Có thể quy ước lớp hộp mang nhãn +1 và lớp nền mang nhãn -1. Các mẫu ảnh hưởng trực tiếp đến biên phân chia được

gọi là vector hỗ trợ. SVM tìm sự cân bằng giữa biên phân chia rộng và việc xử lý các mẫu khó phân tách. [4]

Với dữ liệu không phân tách hoàn toàn, bài toán SVM biên mềm có dạng:

$$\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_i \xi_i \quad (2.7)$$

Các ràng buộc tương ứng là:

$$y_i(\mathbf{w}^\top \mathbf{z}_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0 \quad (2.8)$$

Trong đó ξ_i biểu diễn mức vi phạm biên của mẫu i; C > 0 điều chỉnh mức phạt đối với các vi phạm này. Tham số C cần được lựa chọn trên tập validation. Điểm quyết định SVM không tự động là xác suất hộp. Khi dùng điểm thô của thư viện, phải kiểm tra chiều dấu theo nhãn lớp trước khi áp dụng ngưỡng giữ dự đoán. [5]

2.4.2 Bổ sung mẫu nền khó

Mẫu nền khó là vùng không chứa hộp nhưng được bộ phát hiện đánh giá giống hộp. Quy trình hard negative mining gồm huấn luyện ban đầu, chạy bộ phát hiện trên dữ liệu huấn luyện, thu các vùng nền bị nhận nhầm, bổ sung chúng vào lớp âm và huấn luyện lại. Mẫu chương trình huấn luyện HOG của OpenCV có minh họa bước thu mẫu nhận nhầm từ ảnh âm để huấn luyện lần tiếp theo. [6]

Trong đồ án, cần kiểm tra nhãn trước khi đưa một vùng vào lớp nền. Hộp chưa được gán nhãn hoặc vùng chứa một phần hộp có thể bị chọn nhầm thành mẫu âm. Bổ sung mẫu nền đúng giúp mô hình học các trường hợp gây nhầm lẫn, nhưng hiệu quả phải được đo lại; quy trình này không bảo đảm một mức precision hoặc recall cố định.

2.5 Tạo vùng ứng viên và xử lý khung dự đoán

2.5.1 Quét toàn ảnh và đề xuất vùng trong ROI

Quét cửa sổ trượt đánh giá nhiều vùng ảnh tại các vị trí và kích thước khác nhau. Cách này bao phủ rộng nhưng số lần trích xuất đặc trưng và phân loại có thể lớn. Bước quét thưa làm giảm lượng tính toán, đồng thời có thể bỏ qua vị trí phù hợp hoặc tạo khung bao chưa sát đối tượng.

Một hướng khác là tạo các vùng ứng viên trước khi áp dụng HOG + SVM. Với camera cố định, có thể khai thác ROI, sự khác biệt so với ảnh nền hoặc đặc điểm màu của cảnh. HOG + SVM phân loại các vùng được đề xuất. Vì vậy, chất lượng bộ phát

hiện phụ thuộc cả khả năng tạo ứng viên và khả năng phân loại: đối tượng không được đưa vào danh sách ứng viên sẽ không được SVM kiểm tra.

Cần phân biệt so sánh với ảnh nền tham chiếu và so sánh hai khung hình liên tiếp. Phương án ảnh nền tìm vùng khác nền; phương án hai khung liên tiếp tìm vùng thay đổi theo thời gian. Trong đồ án, các đề xuất dựa trên nền hoặc màu của chương trình PC được đánh giá riêng với khối chênh lệch ảnh xám đang mô phỏng bằng Verilog.

2.5.2 Phép toán hình thái và NMS

Mặt nạ nhị phân có thể chứa các điểm rời rạc hoặc lỗ nhỏ. Phép mở gồm co rồi giãn để loại thành phần nhỏ; phép đóng gồm giãn rồi co để lấp khoảng trống nhỏ. Kích thước phần tử cấu trúc ảnh hưởng đến kết quả, vì xử lý quá mạnh có thể làm mất vùng nhỏ hoặc nối các hộp ở gần nhau. [7]

Sau phân loại, nhiều khung có thể cùng bao quanh một hộp. NMS (Non-Maximum Suppression) chọn khung có điểm cao rồi loại các khung chồng lấn quá mức theo ngưỡng đã đặt. Độ chồng lấn giữa hai khung A và B được đo bằng IoU:

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (2.9)$$

NMS hỗ trợ giảm khung trùng; nó không tự sửa một bộ phân loại sai hoặc bảo đảm giữ được hai hộp sát nhau. Cần phân biệt ngưỡng IoU dùng cho NMS với ngưỡng IoU dùng khi ghép dự đoán với nhãn chuẩn trong đánh giá. [8]

2.6 Chênh lệch ảnh và mặt nạ chuyển động

2.6.1 Quy tắc tính toán

Với hai ảnh xám liên tiếp cùng kích thước và hệ tọa độ, độ chênh lệch tại một pixel được xác định bằng:

$$\mathrm{diff}_t(x, y) = |I_t(x, y) - I_{t-1}(x, y)| \quad (2.10)$$

Đây là phép lấy trị tuyệt đối của chênh lệch từng phần tử, tương ứng với nguyên lý của phép absdiff. [9] Với pixel 8 bit không dấu, diff nằm trong khoảng 0–255. Mặt nạ nhị phân được tạo theo quy tắc của đồ án:

$$\mathrm{motion}_t(x, y) = \begin{cases} 1, & \mathrm{diff}_t(x, y) > T \\ 0, & \mathrm{diff}_t(x, y) \le T \end{cases} \quad (2.11)$$

Trong đó T là ngưỡng nguyên không dấu 8 bit. Quy tắc sử dụng dấu lớn hơn nghiêm ngặt: nếu diff = T thì motion = 0. Với T = 0, mọi chênh lệch khác 0 đều cho

motion = 1; với T = 255, mọi cặp pixel đều cho motion = 0. Ví dụ, hai pixel 150 và 120 có diff = 30; ngưỡng 20 cho motion = 1, còn ngưỡng 30 cho motion = 0. Đây là ví dụ giải thích phép tính, không phải ngưỡng đã tối ưu trên video.

2.6.2 Ý nghĩa và giới hạn

Mặt nạ thể hiện sự thay đổi mức xám, không trực tiếp thể hiện lớp đối tượng. Nhiễu, bóng đổ, thay đổi ánh sáng hoặc rung camera đều có thể tạo điểm thay đổi. Hộp đứng yên có thể không xuất hiện trong mặt nạ; khi hộp di chuyển, mặt nạ cũng có thể chỉ chứa các vùng thay đổi ở biên hoặc một phần bề mặt.

Do đó, mặt nạ chuyển động có thể được nghiên cứu để hỗ trợ lựa chọn vùng xử lý. Nếu sử dụng nó làm điều kiện bắt buộc của bộ phát hiện, cần đánh giá nguy cơ bỏ sót khi hộp chuyển động chậm hoặc dừng. Tính đúng của khối chênh lệch pixel và chất lượng nhận diện hộp là hai yêu cầu khác nhau.

2.7 FPGA và mô tả phần cứng bằng Verilog

2.7.1 Cấu trúc và vai trò của FPGA

FPGA (Field-Programmable Gate Array) là vi mạch có cấu trúc logic có thể cấu hình sau chế tạo. Các tài nguyên cơ bản gồm khối logic, đường kết nối có thể lập trình, bộ nhớ và giao tiếp vào ra. Tệp cấu hình bitstream xác định chức năng và cách kết nối các tài nguyên để hình thành mạch số. [10]

Các thành phần thường gặp gồm LUT (bảng tra thực hiện hàm logic), flip-flop hoặc thanh ghi để lưu trạng thái và bộ nhớ khối. Tùy dòng chip còn có các tài nguyên tính toán chuyên dụng. Thiết kế FPGA mô tả các khối phần cứng có thể hoạt động đồng thời; hiệu năng thực tế phụ thuộc kiến trúc, đường dữ liệu, bộ nhớ và kết quả triển khai. [11]

Trong phạm vi đồ án, phần FPGA tập trung vào một khối tiền xử lý nhỏ gồm lấy trị tuyệt đối của chênh lệch pixel và so sánh ngưỡng. Việc mô phỏng khối này là bước nghiên cứu thiết kế phần cứng. Nó chưa tương đương với triển khai toàn bộ bộ phát hiện hoặc một hệ thống robot trên board.

2.7.2 Mạch tổ hợp và mạch tuần tự

Verilog là ngôn ngữ mô tả phần cứng. Mạch tổ hợp tạo đầu ra từ đầu vào hiện tại, còn mạch tuần tự sử dụng phần tử nhớ để lưu dữ liệu hoặc trạng thái theo clock. Ở mức RTL (Register Transfer Level), thiết kế mô tả các thanh ghi và phép xử lý dữ liệu giữa chúng.

Trong khối chênh lệch pixel, phần tổ hợp thực hiện so sánh hai pixel, chọn thứ tự phép trừ và so sánh diff với ngưỡng. Phần tuần tự lưu kết quả tại cạnh lên của clock. Phép gán không chặn bằng ký hiệu <= được sử dụng trong khối cập nhật thanh ghi; việc lấy giá trị và cập nhật đầu ra phải được xét theo thời điểm clock. [12]

2.7.3 Độ rộng dữ liệu và tránh tràn

Không nên trừ trực tiếp hai số không dấu 8 bit rồi xem kết quả là trị tuyệt đối. Chẳng hạn, 10 − 200 có giá trị toán học −190; nếu chỉ giữ 8 bit không dấu theo phép tính modulo 256 thì giá trị là 66, không phải 190.

Thiết kế có thể so sánh trước để lấy số lớn trừ số nhỏ:

$$\text{diff} = \begin{cases} \text{current} - \text{previous}, & \text{current} \geq \text{previous} \\ \text{previous} - \text{current}, & \text{current} < \text{previous} \end{cases} \quad (2.12)$$

Trong mô-đun của đồ án, hai toán hạng được mở rộng bằng bit 0 thành 9 bit trước phép trừ. Kết quả hợp lệ nằm trong khoảng 0–255 và được lưu bằng 8 bit. Ngưỡng cũng được mở rộng tương ứng khi so sánh. Việc mở rộng bit và chọn đúng thứ tự phép trừ phải cùng tuân theo kiểu dữ liệu không dấu.

2.7.4 Clock reset và tín hiệu hợp lệ

Giao tiếp của khối gồm clock, reset, tín hiệu đầu vào hợp lệ in_valid, hai pixel current và previous, ngưỡng threshold, cùng các đầu ra diff, motion và out_valid. Hai pixel cùng vị trí phải được đưa đến khối đồng thời. Bộ nhớ lưu khung hình và cơ chế ghép pixel nằm ngoài khối đang xét.

Thiết kế sử dụng reset đồng bộ mức cao. Tại cạnh lên clock, reset được ưu tiên và đưa các đầu ra về 0. Khi reset không kích hoạt và in_valid = 1, khối lấy mẫu hai pixel cùng ngưỡng để cập nhật kết quả. Khi in_valid = 0, out_valid được đưa về 0, còn diff và motion giữ giá trị cũ; dữ liệu này không được xem là kết quả hợp lệ mới.

Khối có một tầng đầu ra được đăng ký bằng thanh ghi. Đầu vào ổn định trước cạnh lên thứ k được lấy mẫu tại cạnh đó; đầu ra cập nhật sau cạnh k và có thể được khối tiếp theo lấy mẫu tại cạnh k+1. Đây là quy ước một tầng thanh ghi, thường được mô tả là độ trễ một chu kỳ đối với khối tiêu thụ đồng bộ. Nếu đầu vào hợp lệ liên tiếp, khối có thể nhận một cặp pixel tại mỗi chu kỳ trong mô hình RTL, với điều kiện đáp ứng thời gian phần cứng khi triển khai.

2.8 Edge AI và yêu cầu thời gian thực

Edge AI là cách tổ chức xử lý trí tuệ nhân tạo tại hoặc gần nơi dữ liệu được thu nhận. Trong hệ thống thị giác, suy luận tại thiết bị gần camera có thể giảm nhu cầu truyền toàn bộ dữ liệu đến máy chủ ở xa. Huấn luyện tạo mô hình từ dữ liệu; suy luận sử dụng mô hình đã học để xử lý đầu vào mới. [14]

Đối với đồ án, HOG kết hợp SVM là phương pháp nhận diện được nghiên cứu trên PC, còn mô phỏng Verilog nghiên cứu một khối tiền xử lý có thể phục vụ kiến trúc phần cứng về sau. Việc ghép hai phần, truyền dữ liệu và đánh giá trên thiết bị đích cần được thực hiện riêng trước khi kết luận về một hệ thống Edge AI hoàn chỉnh.

Thời gian thực yêu cầu kết quả được tạo trong giới hạn thời gian phù hợp với ứng dụng. Nếu nguồn ảnh có tốc độ F khung hình mỗi giây thì khoảng thời gian giữa hai khung là:

$$T_{\text{frame}} = \frac{1}{F} (2.13)$$

Ví dụ, nguồn 24 khung hình mỗi giây có khoảng cách giữa hai khung xấp xỉ 41,67 ms. Đây là phép tính minh họa. Với xử lý tuần tự mọi khung, thời gian xử lý và truyền dữ liệu phải phù hợp với ngân sách này để tránh tích lũy hàng đợi; còn hạn chót của ứng dụng có thể phụ thuộc thêm tốc độ băng tải và vị trí cần ra quyết định.

Cần đo riêng độ trễ từ lúc nhận ảnh đến lúc có kết quả, tốc độ xử lý khung hình, mức biến động thời gian và số khung bị bỏ qua. FPS được ghi vào tệp video chỉ quyết định nhịp phát của tệp, không chứng minh tốc độ tính toán. Khả năng nhận một cặp pixel mỗi chu kỳ của khối RTL cũng chưa cho biết FPS của toàn hệ thống vì còn phụ thuộc số pixel, bộ nhớ và các bước nhận diện.

2.9 Đánh giá bộ phát hiện và nguyên tắc sử dụng dữ liệu

2.9.1 Precision recall và F1

Đánh giá phát hiện cần xét đồng thời loại đối tượng và độ khớp của khung bao. Với bài toán một lớp hộp và nhãn đầy đủ, một dự đoán được tính đúng khi ghép với một hộp chuẩn chưa được ghép và đạt ngưỡng IoU đã quy định, chẳng hạn IoU ≥ 0,5. Mỗi hộp chuẩn chỉ được ghép với một dự đoán trong cách đánh giá thông thường.

TP là số dự đoán đúng; FP là số dự đoán sai hoặc khung trùng chưa được ghép; FN là số hộp chuẩn bị bỏ sót. Từ đó:

$$\text{Precision} = \frac{TP}{TP + FP}, \ \text{Recall} = \frac{TP}{TP + FN} \quad (2.15)$$
$$F1 = \frac{2 \ \text{Precision Recall}}{\text{Precision} + \text{Recall}}$$

Precision phản ánh mức đúng trong các khung được dự đoán, còn recall phản ánh mức phát hiện được các hộp có trong nhãn. F1 là trung bình điều hòa của hai chỉ số. Khi mẫu số bằng 0, quy ước xử lý phải được nêu rõ trong chương trình và báo cáo đánh giá. [15]

2.9.2 AP và mAP

AP (Average Precision) tổng hợp đường precision–recall khi xét các dự đoán theo điểm từ cao xuống thấp. AP@0,5, thường viết AP50, sử dụng ngưỡng ghép IoU = 0,5. Nếu dùng nội suy 101 mức recall, cách tính được mô tả bởi:

$$AP = \frac{1}{101} \sum_{j=0}^{100} p_{\text{interp}} \left(\frac{j}{100} \right) \quad (2.16)$$

Trong đó p_interp(r) là precision lớn nhất đạt được ở mức recall không nhỏ hơn r; nếu không có điểm tương ứng thì đóng góp bằng 0. Quy ước 101 mức recall và các ngưỡng IoU từ 0,50 đến 0,95 với bước 0,05 được sử dụng trong công cụ đánh giá COCO. [16]

mAP là trung bình AP theo các lớp đối tượng:

$$mAP = \frac{1}{K} \sum_{c=1}^{K} AP_c \quad (2.17)$$

Với một lớp hộp, mAP bằng AP của lớp hộp tại cùng quy ước đánh giá. Cần ghi rõ AP50 hoặc trung bình trên các ngưỡng IoU 0,50–0,95. AP không phải precision tại một ngưỡng điểm duy nhất. Việc dùng một công thức nội suy giống COCO cũng chưa đồng nghĩa đã thực hiện đầy đủ giao thức COCO, vốn còn có quy định về giới hạn số dự đoán và các nhóm kích thước. [16]

2.9.3 Tách dữ liệu và giới hạn kết luận

Dữ liệu được chia thành train để học mô hình và bổ sung mẫu nền khó, validation để lựa chọn cấu hình, và test để đánh giá cấu hình đã cố định. Các tham số ROI, ngưỡng

tạo ứng viên, ngưỡng SVM và NMS phải được xác định bằng train hoặc validation trước khi đánh giá test. Không sử dụng test để thu mẫu huấn luyện hoặc chọn tham số.

Khi ảnh được trích từ video, các khung gần nhau có thể chứa cùng hộp và nền gần như giống nhau. Việc chia xen kẽ các khung như vậy có thể làm kết quả lạc quan so với khả năng xử lý một video mới. Nếu có thể, nên tách theo video, lần quay hoặc khoảng thời gian có đủ khoảng cách, đồng thời báo rõ mức độc lập của dữ liệu đánh giá.

Kết luận của đồ án cần phân biệt ba vấn đề: chất lượng phát hiện hộp trên dữ liệu có nhãn, thời gian xử lý của chương trình PC và tính đúng của khối tiền xử lý Verilog. Kết quả kiểm chứng số học của khối phần cứng không thay thế precision, recall hoặc AP của bộ nhận diện. Các nội dung robot, bộ nhớ khung hình, giao tiếp camera và hoạt động trên board chỉ được báo cáo là đã triển khai khi có thiết kế và bằng chứng thực nghiệm tương ứng.

TÀI LIỆU THAM KHẢO

[1] OpenCV, “Color conversions”, OpenCV Documentation.

[2] N. Dalal và B. Triggs, “Histograms of Oriented Gradients for Human Detection”, Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), tập 1, 2005, tr. 886–893.

[3] OpenCV, “cv::HOGDescriptor Struct Reference”, OpenCV Documentation.

[4] C. Cortes và V. Vapnik, “Support-Vector Networks”, Machine Learning, tập 20, 1995, tr. 273–297.

[5] F.Iglesias García, “Introduction to Support Vector Machines”, OpenCV Documentation.

[6] OpenCV, “Morphological Transformations”, OpenCV Documentation.

[7] OpenCV, “Deep Neural Network module, mục NMSBoxes”, OpenCV Documentation.

[8] OpenCV, “Operations on arrays, mục absdiff”, OpenCV Documentation.

[9] Altera, “FPGA (Field-Programmable Gate Array)”.

[10] AMD, “7 Series FPGAs Configurable Logic Block User Guide (UG474), mục CLB Overview”, phiên bản 1.9, 2025.

[11] AMD, “Vivado Design Suite User Guide: Synthesis (UG901), mục Using Blocking and Non-Blocking Procedural Assignments”, phiên bản 2024.2, 2024.

[12] S. Williams, “Getting Started With Icarus Verilog”, Icarus Verilog Documentation.

[13] P. Dollár và T.-Y. Lin, “cocoeval.py”, mã nguồn Microsoft COCO Evaluation API, 2015.